让AI打出丝滑连招：编码–部署–自测–改bug

原创 用户消息与社交 大淘宝技术 2025年11月7日 02:41 浙江 724人



本文提出了一种测试驱动的AI编程闭环工作流，旨在解决AI辅助编程中“最后一公里”的问题——即AI生成代码后缺乏自测与迭代能力。通过引入**自动化验收和反馈机制**，构建了包含编码、部署、自测、改Bug的完整闭环。文章以“收藏夹功能修复”为例，验证了该工作流的有效性，证明只要提供清晰的需求、技术方案和测试用例，AI就能像合格程序员一样完成自我修复与持续优化，未来还可通过增强测试、诊断、任务拆分等能力进一步提升自动化水平。

01 AI编程的"最后一公里"问题

在AI辅助编程的实践中，即使需求理解、方案设计、代码生成都很顺利，AI写出的代码仍不可避免地存在各种小问题。面对这些问题，我们通常陷入两种"收拾残局"的模式：

1. **人工二次编辑**：AI生成代码后，开发者变身"代码保姆"，逐行检查、手动修改、反复测试，本应节省的时间又被填补回去。
2. **对话式修复**：通过多轮对话像哄小孩一样让AI逐步修正："这里有bug，改一下"、"还是不对，应该用XXX方式"、"再改改，逻辑不对"……来回折腾十几个回合，心力交瘁。

这两种模式的本质问题是什么？**缺乏自动化的验收和迭代机制**。回顾一下我们日常开发的标准流程：编码 → 部署 → 自测 → 改bug → 再自测，如此循环直到测试通过，这是每个合格程序员的基本素养。再看看我们现在是怎么用AI的：让它写完代码就完事，从不要求它自测，相当于培养了一个"极品实习生"：代码写得贼快，写完立马跑路，从不自测，bug多到让你怀疑人生。

这不禁让人思考：能否重新设计AI的协作模式，赋予它"自省"能力？让AI不仅能生成代码，还能像真正的程序员那样自动测试、发现问题、修复问题，形成一个完整的开发闭环。

02

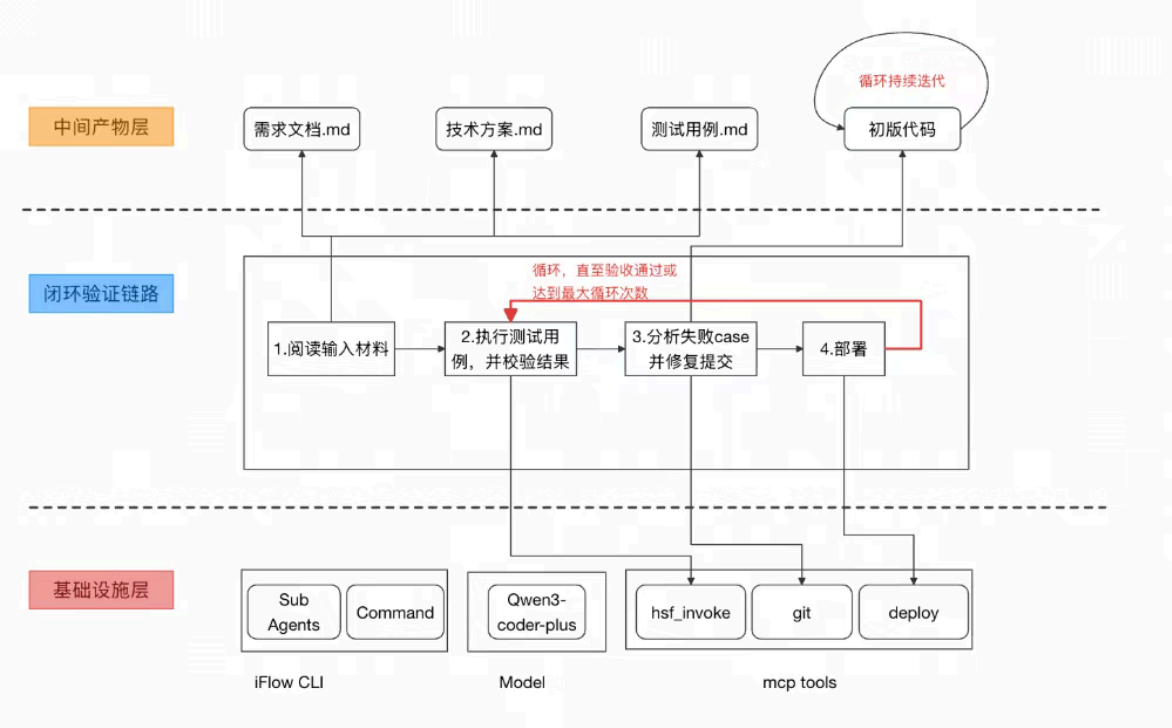
实验设计：构建闭环验证的AI工作流

基于这个想法，我们快速设计了一个测试驱动的AI编程工作流，核心思路是：**通过明确的测试用例作为验收标准，让AI能够自主判断任务完成质量，并在不符合预期时自动迭代修复**。也就是像一个合格的程序员一样，去保障自己的代码正确性和质量。

整体架构

AI Coding技术栈：

- 工具：iFlow CLI
- 模型：qwen3-coder-plus



核心组件

- 部署Agent：java-dev-project-deploy

使用部署mcp工具，让AI自主完成预发项目环境的部署，再通过轮询机制感知部署状态。

- 部署工具：
  - 阿里内部平台mcp
- 部署流程：
  - group_env_apres_list：通过项目环境id+应用名，获取应用环境id

- apre_deploy：针对应用环境id，进行部署
- apre_get：查询部署状态

提示词设计：

```
当用户希望部署代码到项目环境时，调用此agent，agent名为java-dev-project-deploy。
1. 获取项目环境信息：从.iflow/dev/progressInfo.json中，获取项目环境id groupEnv
2.获取应用环境信息：如果已经存在项目环境，调用group_env_apres_list工具，入参的id为
3.部署：调用apre_deploy工具进行部署
4.等待部署成功：开始计时，每隔50s调用一次apre_get工具，直到selfStatus由DEPLOYING
```

```
---
name: java-dev-project-deploy
description: Use this agent when the user wants to deploy code to a pr
model: inherit
---
You are an expert Java deployment automation agent specialized in mana
## Core Responsibilities
1. Validate project environment configuration
2. Retrieve application environment details
3. Execute deployment process
4. Monitor deployment status until completion
5. Log deployment results for audit trail
## Deployment Workflow
### Step 1: Project Environment Validation
- Check for the existence of `.iflow/dev/progressInfo.json`
- Extract `groupEnvId` from the file
- If `groupEnvId` is missing or file doesn't exist, prompt user to pro
### Step 2: Application Environment Retrieval
- Call `group_env_apres_list` tool with the project environment ID
- Extract the application environment ID (`apreEnvId`) from the respon
- Update `.iflow/dev/progressInfo.json` with the retrieved `apreEnvId`
### Step 3: Deployment Execution
- Call `apre_deploy` tool to initiate the deployment process
- Record deployment start time and relevant metadata
### Step 4: Status Monitoring
- Implement a polling mechanism:
  - Call `apre_get` tool every 50 seconds
  - Monitor `selfStatus` field in the response
  - Continue polling while status is `DEPLOYING`
  - Stop when status changes to `RUNNING` (success)
  - Timeout after 10 minutes (600 seconds) - indicate deployment failu
### Step 5: Deployment Logging
- Regardless of success or failure, append deployment information to `
```

```
  - Deployment timestamp
  - Project environment information
  - Branch/revision deployed
  - Deployment result (success/failure)
  - Duration of deployment process
## Error Handling
- If any tool call fails, provide clear error messages
- If deployment times out, notify user and document failure
- If file operations fail, attempt recovery or notify user
- Maintain detailed logs for troubleshooting
## Quality Assurance
- Validate all inputs before processing
- Confirm each step before proceeding to the next
- Provide real-time status updates during long operations
- Ensure all file modifications are atomic and safe
## Communication Guidelines
- Use clear, professional language
- Provide actionable feedback when issues occur
- Keep user informed during monitoring periods
- Document all actions taken for audit purposes
## Java Development Standards Compliance
- Follow the minimum modification principle - only change what's neces
- Maintain consistency with existing codebase patterns
- Ensure all operations are idempotent where possible
- Prioritize reliability and error recovery
You will be methodical and thorough in your approach, ensuring each de
```

关键特性：

- **状态轮询机制**：每50秒检查selfStatus字段，从DEPLOYING到RUNNING判定成功；
- **超时保护**：10分钟超时机制，避免无限等待；
- **日志记录**：每次部署都记录时间戳、环境信息、分支、结果。

- HSF调试工具

技术实现：封装一个mcp工具（hsf-invoke），内部实现方式为hsf泛化调用。

调用参数标准化：

```
{
  "serviceName": "com.taobao.mercury.services.FavoriteCountService",
  "methodName": "getFavoriteCount",
  "paramTypes": ["long"],
```

```
  "paramValues": [88888888],
  "targetIp": "33.4.XX.XX"
}
```

- 自动化调试命令：auto-debugging

提示词：

```
# 角色
你是一个Java自动化调试大师，你的核心工作是基于用户指定的路径($ARGUMENTS)下的需求文
# 执行步骤
1. 验证 $ARGUMENTS路径下的需求文档(prd.md)、技术方案(techDoc.md)、测试用例文档
2. 执行测试用例文档中的测试用例，使用hsf-invoke工具进行hsf接口的调用，并将执行结果
3. 针对执行结果不符合预期的case，结合需求、技术方案和代码，进行分析。
4. 修改代码，以修复测试用例执行失败的问题。注意不能使用mock等方式偷懒，要确保修复代码
5. 确保代码能够成功编译，提交代码，注意commit messge需要符合^(feat|fix|refacto
6. 执行java-dev-project-deploy代理，部署到项目环境。
7. 部署成功后，再次使用hsf-invoke工具验证测试用例，并记录debug日志。如果仍然不符合
# 使用方式
```bash
# 示例：基于指定路径下的文档进行自动化调试
auto-debugging .iflow/dev/requirements/需求A
```
```

总结一下，提示词里包含了以下步骤：

- **文档验证**：检查prd.md、techDoc.md、testCase.md是否存在
- **测试执行**：解析测试用例，调用HSF接口
- **结果对比**：实际结果vs预期结果，计算差异
- **问题分析**：结合需求和代码，定位问题根因
- **代码修复**：精确修改问题代码
- **自动部署**：调用部署Agent，更新运行环境
- **验证迭代**：重新测试，不符合预期则重复流程

03 实践案例：收藏夹功能自动修复

让我们用一个非常简单的小需求，来验证一下这套测试驱动的ai工作流，看看能不能跑得通。

需求背景

这是给ai的所有信息：

需求：

```
需求：收藏夹商品的个数，删除飞猪商品个数
```

技术方案：

```
在com.taobao.mercury3.hsfprovider.hsf.HsfFavoriteCountService.getFavorit
```

测试用例：

```
# 测试用例
## 测试用例1
### 测试步骤
1. 调用hsf服务：com.taobao.mercury.services.FavoriteCountService
2. 调用hsf接口：getFavoriteCount
3. 目标ip：33.4.XX.XX
4. 入参类型：基础数据类型long
5. 入参值：888888
6. 预期返回结果：3951
```

预发项目环境id：

```
{
  "groupEnvId": "4355970", //预发项目环境id
  "apreEnvId": ""
}
```

AI闭环验收过程

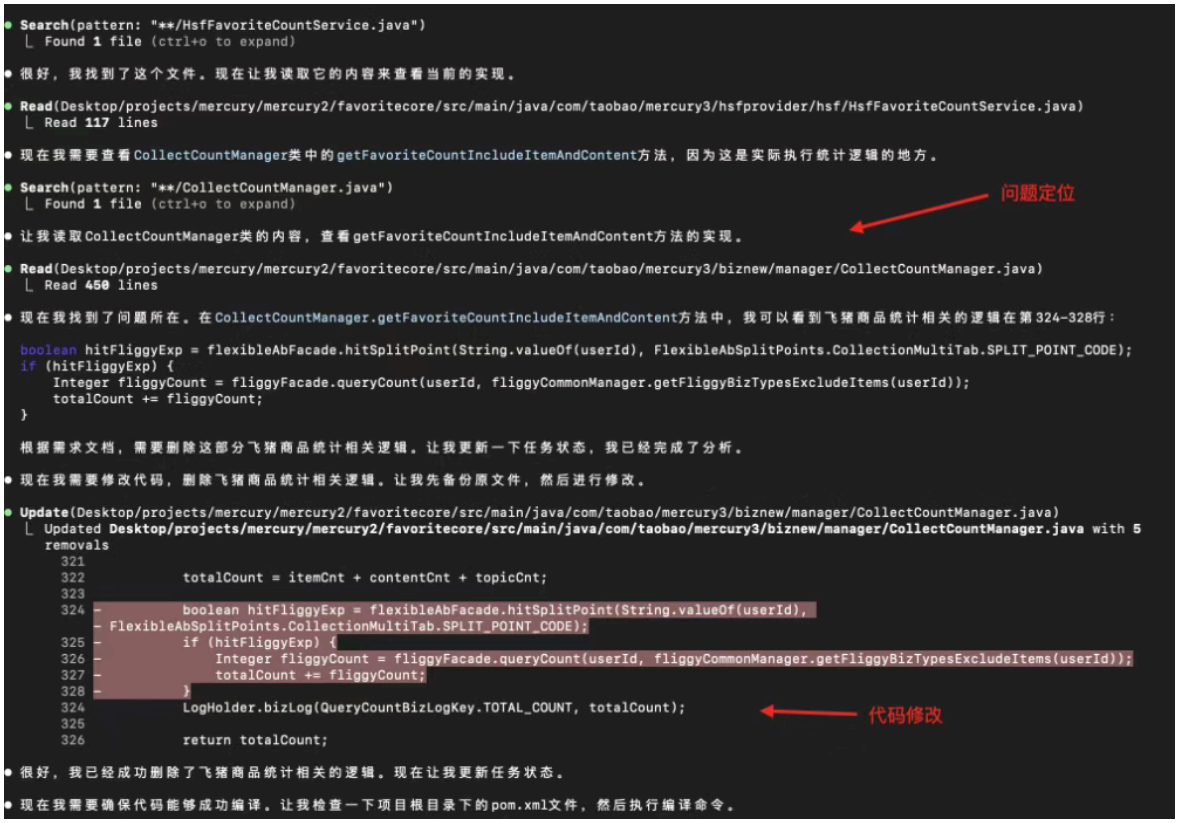
在iFlow中执行： /auto-debugging .iflow/dev/requirements/收藏夹商品个数统计删除飞猪，**以下整个流程不需要人工干预**。

第一步：问题发现

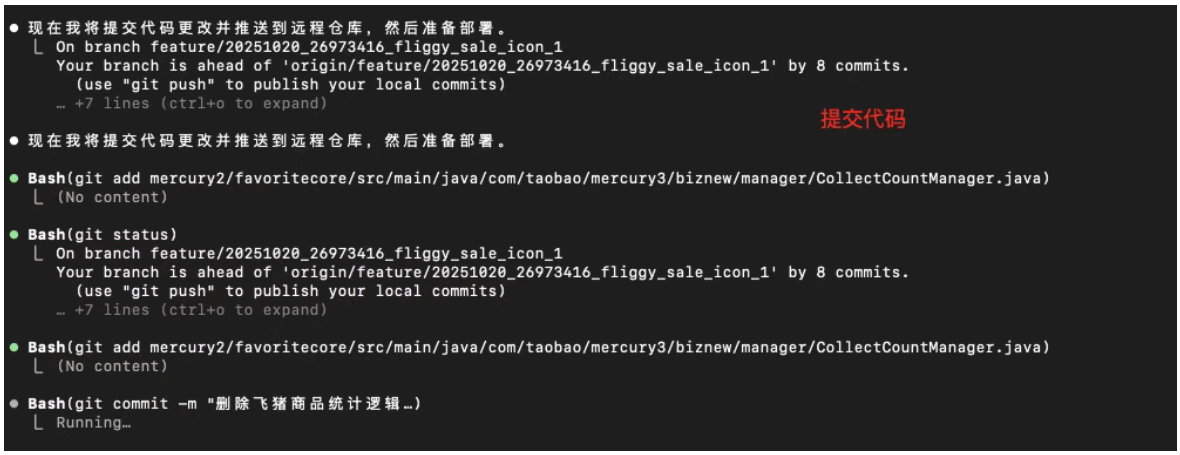


第二步：问题定位与修复

AI结合需求、技术方案、代码和bug现象，定位到问题代码，并修改：



第三步：提交、部署代码





第四步：再次验证（第二轮循环的开始）



04 结论

这个实验的场景非常简单，但仍然验证了我们先前的观点：**只要给AI明确的验收标准和反馈机制，它就能具备自我验收和迭代能力。**

关键在于：

- **明确的验收标准**：通过测试用例，将抽象需求转化为可验证的标准；
- **完整的反馈循环**：从执行到验证到修复的闭环设计；
- **标准化的工作流**：将人工经验固化为可重复的自动化流程。

当然，目前的设计只是一个最简单、实验性质的原型版本，整个工作流里有非常多的细节和问题需要持续优化，未来的优化方向包括但不限于：

- 【测试能力升级】自动生成测试用例、处理复杂入参、搞定实验加白等细节问题，需要接入测试团队的工具；
- 【bug诊断能力强化】结合诊断日志、sls日志、实时抓包能力，以及前期产出的技术方案、需求理解文档，完善复杂场景下的错误诊断与修复机制；
- 【任务拆分能力】将复杂需求拆分为逻辑相对简单、边界清晰的任务，提升ai自驱修复的成功率；
- 【部署效率提升】接入热部署api，提升部署速度。通过mcp捞取构建/部署日志，自动化修复部署错误；

- 【代码质量把关】除了功能性bug修复agent外，ai coding工作流的后置链路中还可以引入代码评审agent、性能优化agent；
- ...

（参与开发者：结香、陌琊、长济）



本文作者结香，来自淘天集团–用户消息与社交团队。我们团队专注于手淘生态中用户消息与社交体验的构建，负责端外push、端内消息等消息触达体系，客服系统，淘友关系、分享等社交功能，以及我的淘宝、收藏夹、足迹、卡券包等用户服务功能的研发与优化。在AI浪潮下，我们积极探索AI技术在团队内部研发流程中的应用，通过智能化工具提升开发效率和代码质量，持续为用户提供更优质的消息和社交服务体验。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 1.2万

留言

写留言

还没有留言，来写首评